Assignment-20.5

K.SAI RAHUL

2303A52392

BATCH-43

Task 1 – Password Storage Vulnerability Check

Analyze an AI-generated signup script for insecure password storage.

Prompt:

Generate a Python user registration and login program that stores user credentials. Then improve it using secure password hashing instead of plain text storage.

Code:

assgn20.5 task1.py > register_user

```python
#Analyze an AI-generated signup script for insecure password storage.
#Prompt AI to generate a Python user registration program.
#Check whether passwords are stored in plain text.
#Improve the script using hashing (bcrypt or hashlib).
import bcrypt
# Function to register user securely
def register_user(username, password):
    # Generate salt and hash password
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

    # Store in file
    file = open("users_secure.txt", "a")
    file.write(username + "," + hashed_password.decode('utf-8') + "\n")
    file.close()

    print("User registered successfully!")
# Function to verify login
def login_user(username, password):
    file = open("users_secure.txt", "r")

    for line in file:
        stored_username, stored_hash = line.strip().split(",")

        if stored_username == username:
            if bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8')):
                print("Login successful!")
                return
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/De
Assist/assgn20.5 task1.py"
User registered successfully!
Login successful!
```

```python
def login_user(username, password):
    file = open("users_secure.txt", "r")

    for line in file:
        stored_username, stored_hash = line.strip().split(",")

        if stored_username == username:
            if bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8')):
                print("Login successful!")
                return
            else:
                print("Incorrect password!")
                return

    print("User not found!")
    file.close()

# Example usage
register_user("john_doe", "securepassword123")
login_user("john_doe", "securepassword123")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-
Assist/assgn20.5 task1.py"
User registered successfully!
Login successful!
```

Observation:

- Password is hashed using bcrypt
- Original password is not stored
- Uses salt automatically
- Even same password → different hash
- Secure against attacks like:
- brute force (harder)
- rainbow table attacks

Task 2 – Hardcoded Secrets Detection

Evaluate AI-generated code for hardcoded API keys or credentials.

Prompt:

Generate a Python script that connects to an external API (OpenWeather) and retrieves weather data. Then evaluate whether API keys are securely handled and improve the code using environment variables instead of hardcoding.

Code:

assgn20.5 task2.py > ...

```python
#Evaluate AI-generated code for hardcoded API keys or credentials.
#Ask AI to generate a script that connects to an external API.
#Identify if API keys/passwords are written directly in code.
#Refactor using environment variables or config files.
#insecure
import requests
def get_weather():
    api_key = "12345ABCDE"   # ❌ Hardcoded secret (insecure)
    url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"

    response = requests.get(url)
    print(response.json())

get_weather()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task2.py"
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
```

assgn20.5 task2.py > ...

```python
#Identify if API keys/passwords are written directly in code.
#Refactor using environment variables or config files.
#insecure
import requests
def get_weather():
    api_key = "12345ABCDE"   # ❌ Hardcoded secret (insecure)
    url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"

    response = requests.get(url)
    print(response.json())

get_weather()

import requests

def get_weather():
    api_key = "98537eb2ffca013b91722703fd910690"   # ✅ your API key

    url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"

    response = requests.get(url)

    print("Status Code:", response.status_code)
    print(response.json())

get_weather()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task2.py"
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
Status Code: 401
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
PS C:\Myfiles\Attachments\Desktop\Ai-Assist>
```

powershell
powershell
powershell
powershell
Python

Observation:

The AI-generated code initially stored API keys directly in the source code, which is insecure and can lead to unauthorized access if the code is shared. Even with a valid key, hardcoding remains a risk. This was improved by using environment variables, which keeps sensitive information outside the code and ensures secure handling of credentials.

Task 3 – File Upload Security Validation

Test an AI-generated file upload program for unsafe file handling.

Prompt:

Generate a Flask-based file upload application. Then analyze it for security issues such as unsafe file handling and improve it by adding validation for file type, size, and secure filename handling.

Code:

assgn20.5 task3.py > upload_file

```python
#Test an AI-generated file upload program for unsafe file handling.
#Prompt AI to generate a Flask file upload example.
#Check for vulnerabilities like uploading executable files.
#Add validation for file type, size, and secure filename handling.
"""from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
    file.save(file.filename)   # ❌ Unsafe
    return "File uploaded!"

if __name__ == '__main__':
    app.run(debug=True)"""


from flask import Flask, request
from werkzeug.utils import secure_filename
import os

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task3.py"
 * Serving Flask app 'assgn20.5 task3'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -
```

```python
app = Flask(__name__)

# Upload folder
UPLOAD_FOLDER = 'uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

# Allowed file types
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'pdf'}

# Max file size (2MB)
app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024


# Check file type
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS


# Home route (IMPORTANT)
@app.route('/')
def home():
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task3.py"
 * Serving Flask app 'assgn20.5 task3'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -
```

```python
# Upload route
@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return "No file part"

    file = request.files['file']

    if file.filename == '':
        return "No selected file"

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        file.save(os.path.join(UPLOAD_FOLDER, filename))
        return "File uploaded successfully!"

    return "Invalid file type!"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachm
Assist/assgn20.5 task3.py"
 * Serving Flask app 'assgn20.5 task3'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -
```

Upload File

Choose File 17.1 Assignment.pdf Upload

File uploaded successfully!

Observation:

The AI-generated file upload code was initially insecure as it allowed unrestricted file uploads, used unsafe filenames, and lacked validation, making it vulnerable to attacks like uploading malicious files. The improved version adds file type restrictions, size limits, and uses secure filename handling, ensuring safe and controlled file uploads.

Task 4 – Command Injection Prevention

Analyze an AI-generated Python script that executes system commands.

Prompt:

Generate a Python script that executes system commands using user input. Then analyze it for command injection vulnerabilities and improve it using safe subprocess methods and input validation.

Code:

assgn20.5 task4.py > ...

```python
#Analyze an AI-generated Python script that executes system commands.
##Ask AI to generate code using os.system() or subprocess.
#Identify command injection risks with user input.
#Fix using safe subprocess calls and input validation.
import os

def run_command():
    user_input = input("Enter a command: ")
    os.system(user_input)   # ❌ Dangerous

run_command()

import subprocess

def run_command():
    filename = input("Enter file name: ")

    if not filename.replace('.', '').isalnum():
        print("Invalid input!")
        return

    try:
        result = subprocess.run(
            ["dir", filename],
            capture_output=True,
            text=True,
            check=True
        )
        print(result.stdout)
    except subprocess.CalledProcessError:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/as
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/as
Enter a command: dir
 Volume in drive C is Windows
 Volume Serial Number is A4AC-9906

 Directory of C:\Myfiles\Attachments\Desktop\Ai-Assist

02-04-2026  14:22    <DIR>          .
25-03-2026  22:10    <DIR>          ..
01-04-2026  09:21               387 1.sql
23-03-2026  14:45             5,130 assgn14.1 task1.html
23-03-2026  14:43             2,169 assgn14.1 task2.html
```

assgn20.5 task4.py > ...

```python
def run_command():
        print("Invalid input!")
        return

    try:
        result = subprocess.run(
            ["dir", filename],
            capture_output=True,
            text=True,
            check=True
        )
        print(result.stdout)
    except subprocess.CalledProcessError:
        print("Error executing command")

run_command()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/ass
02-04-2026  14:22    <DIR>          .
25-03-2026  22:10    <DIR>          ..
01-04-2026  09:21               387 1.sql
23-03-2026  14:45             5,130 assgn14.1 task1.html
23-03-2026  14:43             2,169 assgn14.1 task2.html
23-03-2026  14:43             4,577 assgn14.1 task3.html
23-03-2026  14:45             3,367 assgn14.1 task4.html
23-03-2026  14:47             2,559 assgn14.1 task5.html
24-03-2026  11:53               275 assgn15.4 task1.py
25-03-2026  09:17               656 assgn15.4 task2.py
24-03-2026  22:07             1,353 assgn15.4 task3.py
24-03-2026  22:13             1,717 assgn15.4 task4.py
```

```python
def run_command():
        print("Invalid input!")
        return

    try:
        result = subprocess.run(
            ["dir", filename],
            capture_output=True,
            text=True,
            check=True
        )
        print(result.stdout)
    except subprocess.CalledProcessError:
        print("Error executing command")

run_command()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assi
25-03-2026  09:17              656 assgn15.4 task2.py
24-03-2026  22:07            1,353 assgn15.4 task3.py
24-03-2026  22:13            1,717 assgn15.4 task4.py
25-03-2026  09:17            3,149 assgn15.4 task5.py
01-04-2026  09:21            1,229 assgn16.4 task1.sql
01-04-2026  09:21            1,364 assgn16.4 task2.sql
01-04-2026  09:21            1,294 assgn16.4 task3.sql
01-04-2026  09:21            1,230 assgn16.4 task4.sql
01-04-2026  09:21            1,384 assgn16.4 task5.sql
30-03-2026  22:13              763 assgn17.1 task1.py
01-04-2026  09:21            1,004 assgn17.1 task2.py
30-03-2026  22:44            1,356 assgn17.1 task3.py
```

Observation:

The AI-generated code initially used os.system() with direct user input, making it vulnerable to command injection attacks where malicious commands can be executed. The improved version replaces it with subprocess.run() without using a shell and adds input validation to restrict unsafe characters. This ensures that only safe commands are executed and prevents unauthorized system access.

Task 5 – Secure Input Handling in Web Forms

Check AI-generated web form code for improper input validation.

Prompt:

Generate an HTML and Flask-based feedback form. Then analyze it for improper input validation and unsafe rendering, and improve it using server-side validation and input sanitization.

Code:

templates > form.html > form

```html
<form method="POST" action="/submit">
    <h2>Feedback Form</h2>

    Name: <input type="text" name="name" required><br><br>

    Feedback:<br>
    <textarea name="feedback" required></textarea><br><br>

    <input type="submit" value="Submit">
</form>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task.py"
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task5.py"
 * Serving Flask app 'assgn20.5 task5'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 784-284-462
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [03/Apr/2026 12:21:23] "POST /submit HTTP/1.1" 200 -
```

assgn20.5 task5.py > ...

```python
#Check AI-generated web form code for improper input validation.
#Ask AI to generate an HTML + Flask feedback form.
# Identify missing validation and unsafe rendering.
#Fix using server-side validation and sanitization.
from flask import Flask, render_template, request
from markupsafe import escape

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('form.html')

@app.route('/submit', methods=['POST'])
def submit():
    name = request.form.get('name', '').strip()
    feedback = request.form.get('feedback', '').strip()

    # Validation
    if not name.isalnum():
        return "Invalid name!"

    if len(feedback) > 200:
        return "Feedback too long!"

    # Sanitization
    name = escape(name)
    feedback = escape(feedback)

    return f"Thank you {name}, your feedback: {feedback}"

if __name__ == '__main__':
    app.run(debug=True)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task5.py"
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 784-284-462
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [03/Apr/2026 12:21:23] "POST /submit HTTP/1.1" 200 -
```

Observation:

The AI-generated form initially lacked proper input validation and directly rendered user input, making it vulnerable to injection attacks like XSS. The improved version adds server-side validation to restrict invalid inputs and uses escape() to sanitize user data before displaying it. This ensures that malicious scripts cannot be executed and the application handles user input securely.